UNIVERSIDADE REGIONAL DE BLUMENAU
CENTRO DE CIÊNCIAS EXATAS E NATURAIS
DEPARTAMENTO DE SISTEMAS E COMPUTAÇÃO
PROFESSOR ANDRÉ FELIPE BÜRGER
PROGRAMAÇÃO ORIENTADA A OBJETOS

Trabalho Final

GiroBlu: bicicletas e patinetes compartilhados

O CONTEXTO

A GiroBlu é uma startup de Blumenau que quer espalhar bicicletas e patinetes compartilhados pela cidade. A ideia é a mesma dos serviços que você já viu em outras capitais: o usuário chega a uma estação, libera um veículo, vai até onde precisa e devolve em qualquer estação que tenha vaga. Tudo é pago com uma carteira de créditos pré-paga.

Antes de lançar o aplicativo para o público, a GiroBlu precisa de um sistema de retaguarda para a equipe de operação: cadastrar a frota, acompanhar as viagens, cobrar os usuários e saber quanto entrou de dinheiro. É esse sistema que você vai construir, em Java, com uma interface gráfica em Swing.

Repare que este enunciado descreve o negócio, mas **não diz quais classes você deve criar**. Isso é proposital: interpretar o problema e decidir a modelagem faz parte do trabalho. Equipes diferentes vão chegar a diagramas diferentes, e tudo bem, desde que os resultados (valores cobrados, cargas de bateria, saldos) obedeçam às regras abaixo. Na apresentação, você vai defender as escolhas que fez. Ficou com dúvida sobre alguma regra de negócio? Pergunte em aula, como faria com um cliente de verdade.

COMO A OPERAÇÃO FUNCIONA

Estações

A cidade terá várias estações, cada uma com um nome (Prainha, Vila Germânica, FURB Campus 1...) e uma quantidade de vagas. Uma estação nunca pode guardar mais veículos do que suas vagas. Veículos parados, inclusive os que estão em manutenção, ocupam vaga; veículos em uso, não.

Veículos

A frota começa com três tipos de veículo: bicicleta mecânica, bicicleta elétrica e patinete elétrico. Todo veículo tem um código de identificação, um modelo e fica em uma estação até alguém usá-lo. A cada momento, um veículo está disponível, em uso ou em manutenção, e só veículos disponíveis podem ser liberados para uma viagem. Qualquer veículo que não esteja em uso pode ser enviado para manutenção e, depois, voltar a ficar disponível.

Os veículos elétricos têm bateria, medida em percentual. Um veículo elétrico só pode ser liberado com pelo menos 20% de carga, e cada minuto de uso consome bateria: 1% por minuto na bicicleta elétrica e 2% por minuto no patinete. A carga nunca fica abaixo de 0%. A equipe de operação pode recarregar um veículo elétrico (ele volta a 100%) desde que ele não esteja em uso. A bicicleta mecânica, naturalmente, não tem nada disso.

A GiroBlu já avisou que outros tipos de veículo devem chegar no futuro (quem sabe um triciclo de carga). O sistema precisa permitir incluir um tipo novo mexendo o mínimo possível no código que já existe.

Cada tipo de veículo tem sua tarifa: uma taxa de desbloqueio, cobrada uma vez por viagem, mais um valor por minuto de uso.

Veículo	Desbloqueio	Por minuto
Bicicleta mecânica	R$ 1,00	R$ 0,15
Bicicleta elétrica	R$ 2,00	R$ 0,30
Patinete elétrico	R$ 3,00	R$ 0,40

Usuários, planos e carteira

Cada usuário tem nome, e-mail, um plano e uma carteira de créditos. A carteira começa zerada e é abastecida por recargas; cada recarga tem data, hora e um valor, que precisa ser positivo. Hoje existem três planos:

- **Avulso:** paga o preço cheio da tabela.
- **Mensal:** não paga a taxa de desbloqueio, e os primeiros 15 minutos de cada viagem são gratuitos (paga só os minutos que passarem disso).
- **Universitário:** tem 50% de desconto sobre o valor total da viagem, exceto em patinetes, que são cobrados pelo preço cheio.

Assim como acontece com os veículos, um plano novo deve poder entrar no sistema sem que seja preciso reescrever as regras dos planos que já existem.

Viagens

Uma viagem começa quando um usuário libera um veículo em uma estação, informando a data e a hora de início. Para começar, o usuário não pode ter outra viagem em andamento e precisa ter pelo menos R$ 5,00 de saldo na carteira; o veículo precisa estar disponível e, se for elétrico, com carga suficiente.

A viagem termina quando o veículo é devolvido em uma estação com vaga livre, informando a data e a hora de término (que precisa ser posterior ao início). Nesse momento o sistema calcula a duração em minutos e o valor a pagar, de acordo com o veículo e com o plano do usuário, arredondado para duas casas decimais. O valor é debitado da carteira e o veículo passa a estar na estação de destino, disponível e, se for elétrico, com a bateria descontada.

O débito acontece mesmo que o saldo fique negativo. Nesse caso, o usuário só consegue iniciar uma nova viagem depois de fazer recargas suficientes.

Alguns exemplos para você conferir as suas regras:

Situação	Valor cobrado	Bateria final
Carla (Avulso), bicicleta mecânica, 12 minutos	R$ 1,00 + 12 × R$ 0,15 = R$ 2,80	-
Ana (Universitário), bicicleta elétrica com 80%, 25 minutos	(R$ 2,00 + 25 × R$ 0,30) × 50% = R$ 4,75	55%
Bruno (Mensal), patinete com 60%, 20 minutos	5 minutos × R$ 0,40 = R$ 2,00	20%
Diego (Universitário), patinete com 100%, 10 minutos	R$ 3,00 + 10 × R$ 0,40 = R$ 7,00	80%
Eva (Mensal), bicicleta mecânica, 10 minutos	R$ 0,00	-

O QUE O SISTEMA DEVE PERMITIR

1. Cadastrar estações, veículos (de cada tipo, já posicionados em uma estação) e usuários (com o plano escolhido).
2. Registrar recargas na carteira de um usuário.
3. Iniciar uma viagem.
4. Encerrar uma viagem, mostrando a duração e o valor cobrado.
5. Enviar um veículo para manutenção, devolvê-lo à operação e recarregar a bateria de veículos elétricos.
6. Consultar os veículos disponíveis em uma estação, mostrando o tipo de cada um e, para os elétricos, a carga atual.
7. Emitir o extrato de um usuário: todas as recargas e viagens em ordem cronológica (a viagem entra no extrato na data e hora em que foi encerrada), mostrando, a cada lançamento, o valor como crédito ou débito e o saldo da carteira depois dele, como um extrato bancário.

8. Consultar o faturamento de um período informado (data inicial e data final): quantidade de viagens encerradas no período e valor arrecadado, separados por tipo de veículo, e o total geral.

Sempre que uma regra for violada (estação lotada, saldo insuficiente, veículo indisponível, datas inválidas...), o sistema deve mostrar uma mensagem clara ao usuário, sem travar e sem perder dados.

O QUE PRECISA APARECER NO SEU PROJETO

Este é o trabalho que fecha a disciplina, então ele precisa reunir tudo o que vimos no semestre. A escolha de onde aplicar cada item é sua, mas todos precisam estar lá, e você vai mostrar onde e por que na apresentação.

- **Arquitetura em duas camadas:** camada de apresentação (Swing) e camada de negócio. A camada de negócio não conhece nada de Swing, e nenhuma regra de negócio fica dentro das telas.
- **Encapsulamento:** o estado dos objetos só muda por métodos que garantem as regras. Ninguém altera a bateria de um veículo ou o saldo de uma carteira "por fora".
- **Herança, classe abstrata e método abstrato:** pelo menos uma hierarquia com uma superclasse abstrata que declare pelo menos um método abstrato.
- **Polimorfismo:** o que varia conforme o tipo de objeto deve ser resolvido por sobrescrita de métodos, e não por cadeias de if ou instanceof perguntando "que tipo de objeto é esse?".
- **Interface:** pelo menos uma interface criada pela equipe e implementada por mais de uma classe. Implementar interfaces prontas da API (ActionListener, Comparable, Serializable...) é permitido, mas não conta para este item.
- **Exceções:** violações de regra de negócio sinalizadas com exceções, com pelo menos uma exceção criada pela equipe, e tratadas na interface gráfica com uma mensagem ao usuário.
- **Persistência em arquivo:** os dados são gravados em disco e recuperados automaticamente quando o programa é iniciado, em arquivos CSV (.csv) ou binários. Fechar e abrir o programa não pode perder nada, nem as viagens em andamento.
- **Interface gráfica simples:** rótulos, campos de texto e botões bastam. Os resultados das consultas (veículos, extrato, faturamento) podem ser exibidos em um campo ou área de texto, sem necessidade de tabelas. O visual não vale nota; o que importa é a tela funcionar e conversar corretamente com a camada de negócio.
- **Coleções:** liberadas (ArrayList e companhia). Bibliotecas de terceiros não são permitidas, com exceção do JUnit.

MÉTODO DE AVALIAÇÃO

Pontos	Item avaliado
1,5	Diagrama de classes UML da camada de negócio, com todas as classes, interfaces e relacionamentos, compatível com o código-fonte. Ferramenta livre (sugestão: draw.io).
4,5	Implementação: requisitos funcionais 1 a 8 funcionando conforme as regras (2,0); conceitos de POO aplicados como pedido acima (1,5); persistência automática (0,5); interface gráfica separada da camada de negócio e javadoc (0,5).
0,5	Plano de testes para validar as classes da camada de negócio.
1,5	Implementação do plano de testes com JUnit. Todos os métodos públicos da camada de negócio que não sejam getters e setters devem ter testes.
2,0	Apresentação (nota individual, detalhada abaixo).
10,0	Total

APRESENTAÇÃO

Cada equipe tem **no máximo 10 minutos**: cerca de 8 minutos de apresentação e 2 minutos para perguntas. A apresentação deve seguir obrigatoriamente o **template disponibilizado no AVA**, que já traz os slides na ordem esperada:

Slide	O que mostrar	Tempo
Capa	Equipe	0:15
Diagrama de classes	Visão geral do diagrama da camada de negócio	1:30
Decisões de modelagem	As principais escolhas da equipe e as alternativas descartadas	1:00
Onde usamos cada conceito	Encapsulamento, herança, classe e método abstrato, polimorfismo e interface: onde e por quê	1:30
Polimorfismo em ação	Um trecho curto de código que mostre o polimorfismo funcionando	1:00
Plano de testes	Uma amostra dos casos de teste	0:45
Testes implementados	Um ou dois métodos JUnit que implementam casos do plano	0:45
Execução dos testes	Resultado da execução dos testes unitários	0:30
Como foi aplicar os conceitos	Dificuldades, aprendizados e o que fariam diferente	0:45

Todos os integrantes devem apresentar pelo menos um slide, e qualquer integrante pode ser chamado a explicar qualquer parte do código. A nota da apresentação é individual:

Pontos	Critério
0,5	Clareza do diagrama e justificativa das decisões de modelagem.
0,5	Explicação de onde e por que cada conceito de POO foi aplicado.
0,4	Plano de testes, testes implementados e resultado da execução.
0,4	Domínio do código demonstrado nas respostas às perguntas.
0,2	Uso do template e respeito ao tempo.

MATERIAL A SER SUBMETIDO NO AVA

- O projeto com os arquivos-fonte (camadas de negócio e de apresentação e os testes JUnit), compactado em .zip.
- Os arquivos de dados com alguns registros de exemplo, para o sistema já abrir com informações.
- O diagrama de classes UML, em PDF.
- O plano de testes, em PDF.
- A apresentação, feita no template.

RESTRIÇÕES E PRAZO

O trabalho deve ser feito em equipe de no máximo 3 pessoas.

- Entrega no AVA até **29/11/2026 às 23:59**.
- Apresentações nas duas últimas aulas: **Turma 001 em 30/11 e 03/12**; **Turma 004 em 01/12 e 03/12**.
- Conforme o plano de ensino, soluções copiadas de outras equipes, da internet ou geradas por ferramentas de inteligência artificial recebem nota zero no item avaliado.